Dijkstra’s shortest-path algorithm

Implementation baseline - bounded layout, retry semantics and stable scene rebuilding

svg-dom-graph v0.2.40 | Incorporates implementation review, 22 October 2026 | Proposal only

Review resolution. Verified public Scene::move_node and self-loop rejection. This baseline also pins a label-placement scoring threshold, defines when stale DOM writes are retried, adopts the existing Huffman/Hamming/CRC-32 SVG rebuild pattern, and reserves an independently anchored priority-queue region.

1. Design decision

The algorithm remains in a pure Rust demo module; **svg-dom-graph** only renders the state. No Dijkstra-specific public API types are proposed. Existing Boolean arrays, selection, node focus, text/Markdown descriptions and graph nodes are reused.

One reusable data-value extension is proposed: **NodeValues::Text(Vec<String>)**, displayed with **DataFormat::Text**. Two reusable connector features are proposed: edge labels and per-edge focus. A small general-purpose cell-width reservation is included to prevent changing values from moving the layout.

Addition	Scope	Decision
Text-valued data cells	Library	Required; independently selectable Unicode strings
Cell-width hint	Library	Required for stable demo geometry
Edge labels	Library	Required; positioned using computed route points
Per-edge focus	Library	Required; bool per EdgeId, multiple simultaneous
Dijkstra algorithm and snapshots	Demo	Required; no domain logic in library

2. Executable model and display

For an initial six-vertex directed graph, store integer weights and distances exactly:

```
struct WeightedEdge { from: usize, to: usize, weight: u64 }
struct AlgorithmState {
    distance: Vec<Option<u64>>,
    predecessor: Vec<Option<usize>>,
    settled: Vec<bool>,
    // Queue and step-specific state are demo-owned.
}
```

None means unreachable/unknown, rendered as ∞. It is never encoded as u64::MAX. Relaxation uses checked_add; overflow is reported. Reject negative weights before starting. Keep an option for an unreachable target and describe it explicitly in the final narration.

The demonstration presents the graph separately from aligned one-column arrays labelled Distance, Previous and Settled, and an explanatory priority-queue view. Row labels identify vertices, so a predecessor such as “B” or “—” cannot be confused with the row’s own vertex.

2.1 Verified reference graph

Directed edge	Weight	Directed edge	Weight
A → B	4	A → C	2
C → B	1	B → D	5
C → D	8	C → E	10
D → E	2	D → F	6
E → F	3		

From source A, final distances are A=0, B=3, C=2, D=8, E=10, F=13. Predecessors are A=—, B=C, C=A, D=B, E=D, F=E. The shortest route to F is **A → C → B → D → E → F**, of weight **13**.

2.2 Reachable intermediate state (corrected)

The earlier example incorrectly combined D=8 with E=∞. The following state is reachable **immediately after B is settled and B→D is relaxed**; C was settled earlier and already relaxed C→E.

Vertex	Distance	Previous	Settled
A	0	—	Yes
B	3	C	Yes
C	2	A	Yes
D	8	B	No
E	12	C	No
F	∞	—	No

Before B is settled, C→D has established D=10 via C. B→D improves D to 8 via B; E is already 12 via C, and does not become 10 via D until D is settled.

3. Text-cell extension

3.1 Public API

```
// Proposed additions (not yet implemented):
NodeValues::Text(Vec<String>)
DataFormat::Text
DataNodeContent::with_cell_width_hint(&str)
```

Each string is one independently selectable cell, with the existing GridLayout, selection, secondary selection, labelled rows and accessibility semantics. Text uses SVG text-node APIs only; never interpret it as markup. Reject embedded line breaks and unsupported controls in the initial version. Empty strings are permitted and described accessibly as “empty”.

Format compatibility is explicit: Text values require DataFormat::Text; other NodeValues variants do not accept this format. Values remain homogeneous by node, preserving the current model.

3.2 Implementation consideration: non-Copy values

The current numeric formatting implementation has a Copy-bound Cell trait. String is not Copy. Handle the Text variant through a dedicated borrowed-string formatting/measurement path; do not weaken or replace the existing numeric fast path merely to accommodate strings. Avoid per-cell intermediate String allocation during display, update or accessibility construction.

Measure Unicode strings from rendered glyph bounds, not character counts. Test creation, replacement and accessible labels, even though Dijkstra rebuilds instead of using set_data_values during stepping.

3.3 Stable cell widths: decided now

Decision: The demo rebuilds its Scene from an immutable snapshot on every step, and does not use set_data_values for step transitions. Still, node and edge positions must not jump when “∞” changes to “1250”. Therefore add a generic cell-width hint during construction.

The hint is a sample string measured using the cell font. Each column’s width is at least the measured hint width, but wider real contents still win: the hint alone does **not** guarantee stability. Compute the widest actual displayed string across **every precomputed snapshot** for each data column, measure it, and use it as the hint. This includes ∞, — and vertex names. The hint contributes no value, selection or accessible narration.

Stability acceptance: for a fixed input graph, the positions **and dimensions** of vertex boxes and Distance, Previous and Settled nodes are identical across all step snapshots (before intentional user drags). The priority queue has a **fixed, separately anchored rectangle**; no change in its rows or message text may move the graph. Verify invariant node rectangles for every precomputed step. Recompute widest actual strings when input changes. LabellingStyle::Text already accepts static row labels for A–F.

4. Edge-label API and geometry

4.1 Public API and ownership

```
// Proposed: attach to an existing EdgeId.
scene.set_edge_label(edge, Some("8"))?;
scene.set_edge_label(edge, None)?;  // remove label
```

The label is a **separate SVG group** owned by ConnectorHandle, not a vertex and not a text child of the connector path. The existing handle holds a path and optional L/R port marker; add an optional EdgeLabelHandle for text, background, cached text extent and chosen placement. ConnectorOptions remains Copy and must not acquire a String. The group lives under the same scene content layer as the path so pan/zoom moves them together without re-routing.

Create an SVG <g> with a background rect and text, role=group and an accessible name identifying endpoints and weight. Set a stable data-svg-dom-graph-edge-label marker, scoped by scene and edge index for CDP tests. Background fill/ink must reach **4.5:1 normal-text contrast**; a non-colour-only active-edge treatment must have **3:1 non-text contrast** where WCAG applies. Put a simple native colour-ratio test beside the palette tests; do not restore a broad browser colour suite.

4.2 Anchor to actual route geometry

The existing connector::route returns 2–4 route points plus corner radius. The label placer must use that result directly, in the same call that writes the connector path. Do not parse the path d attribute or read it back from the DOM.

For each straight segment between route points, determine its **usable** interval by trimming rounded corners. At an interior corner, the trim is min(requested radius, incoming length/2, outgoing length/2), matching elbow_path_into. Also reserve clearance near arrowheads and endpoint node boxes. Prefer a long segment, ordinarily away from the source when several routes emerge together.

Use a **fixed, small candidate set** (maximum eight candidates per label): first the longest usable segment at its midpoint, then alternative fractions and opposite-side offsets. Candidate background rectangles must clear the connector stroke and arrowhead. For horizontal runs prefer above; for vertical runs prefer right. Deterministic scores and EdgeId tie-breaking govern candidate choice.

4.3 Collision behaviour

Build the label placer as a **pure DOM-free function** over route points, measured label rectangles, vertex boxes, arrowhead regions and prior placements. Each candidate checks intersections with expanded vertex boxes, the edge’s own other segments, arrowheads, and nearby accepted label rectangles. Do not perform getBBox/text measurement or parse SVG path d on a drag frame. Keep candidate count at eight or fewer.

Bounded work: route and re-place the edges affected by the moved node, plus labels in the local collision area. Use broad-phase bounding boxes or a spatial index to avoid scanning all vertices and all labels for every candidate on every drag frame. Process EdgeId in stable order; do **one initial placement and at most one forward reconciliation pass**. Earlier accepted placements have priority; only lower-priority conflicting labels may move. Do not iterate until convergence.

Deterministic placement score (pin in placer doc comment): each candidate has a hard-conflict tuple (number of blocking overlaps, total overlap area) and a soft integer score: **round(abs(dx) + abs(dy)) + 16 for a non-preferred side**, where dx and dy are SVG user-space offsets from the preferred segment anchor. Rank collision-free candidates first; for fallback conflicts rank by overlap count, overlap area, soft score, then candidate ordinal. Use EdgeId for cross-label ordering. A still-valid previous candidate index remains selected unless the best valid alternative improves soft score by **at least 12 points**. Recompute its coordinates from the new route, never freeze the old absolute position. A newly invalid candidate may switch regardless of threshold. Test 11-point improvement (retain), 12-point improvement (switch), and tie cases.

No oscillation and termination: at most one forward reconciliation pass after initial placement, with higher-priority EdgeIds fixed and only lower-priority conflicting labels eligible to move. Never restart reconciliation or iterate until convergence. If both passes leave conflicts, keep the deterministic least-bad result and stop; do not hide weights. The reference demo must be collision-free at rest and for bounded test drags, but arbitrary crowded graphs may retain documented overlaps. Specifically test C→D (8) against C→E (10); adjust demo vertex geometry or add a small Copy-based placement hint if needed.

4.4 Drag updates and CDP hooks

Routes change through **public Scene::move_node** (src/scene/node/mod.rs:298), the internal drag path using SceneInner::move_node, set_edge_anchors, set_connector_type and operator-input reroutes. Recompute path and label from **one new Route** and apply them in the same coalesced frame. After any successful update, path and label geometry agree. SVG writes are **not atomic**: if a path or label write fails, mark the edge route/label pair stale and surface the failure through the calling API or existing demo error reporting. Do not clear stale until both writes succeed.

Defined retry point: the **next route update for that EdgeId** reattempts both path and label writes even if its geometry or options are unchanged. On pointer release after a drag, perform a final flush/retry for stale incident edges, even if no pointer moved in that frame. A repeat call to the edge-label setter with the same text also retries a stale edge; unchanged-value fast paths must check the stale flag. Do not schedule unbounded timers or recursive retries. With no subsequent event, a failed edge remains stale and the error remains reported.

CDP tests should select data-svg-dom-graph-edge-label with scene-local edge identity and compare its rectangle with the route/vertex rectangles; do not depend on child order. Pure fault-injection tests must assert that a failed second write leaves stale=true and the next route update (or drag completion) retries and clears it only after both succeed. Also assert ≤8 candidates, ≤2 passes, cached extents and skipped no-op writes.

5. Edge focus and state coexistence

```
// Proposed: independent per-edge state.
scene.set_edge_focus(edge_id, true)?;
scene.set_edge_focus(edge_id, false)?;
```

Focus is stored per EdgeId; it must never be implemented as a single scene-wide “current edge”. Any number of edges may remain focused simultaneously for the final path. Its visual treatment changes stroke width and/or emphasis without changing routes, endpoints, labels or the caller’s algorithm state.

The node and edge focus mechanisms operate on different SVG handles and therefore do not override each other. The demo uses plain graph nodes with set_focus and separate data nodes with set_selection; it must not apply both node focus and data selection to the same rendered rect, given the currently documented styling interaction.

For an active relaxation, emphasise one edge and its source/target nodes; for the final path, emphasise all predecessor-chain edges at once. Use stroke pattern/width and accessible step narration (e.g., “examining C to E, weight 10; candidate 12”) rather than colour alone. Palette tests must verify label text-background contrast and relevant focus-state contrast.

6. Snapshot-driven interaction and view preservation

Implementation order changes: Build the pure algorithm and its tests first. Do not build a disposable scene using float infinity, ASCII predecessor arrays and static weight text. Once text cells and edge labels exist, build the actual Dijkstra scene directly.

Each immutable step snapshot contains displayed distances, predecessors, settled flags, queue entries, active vertex, current examined edge, relaxation result, and narration. Previous and Restart select a snapshot; they do not reverse binary-heap operations. Ties are deterministic: order the queue by (distance, vertex index); on equal candidate distance retain the existing predecessor. For parallel equal-cost edges, preserve insertion order or explicit edge index in the trace.

Use the existing rebuild pattern found in demo-app/src/huffman/mod.rs, demo-app/src/hamming/mod.rs and demo-app/src/crc32/mod.rs; **do not introduce a separate root-swap abstraction**. Capture Scene::view(), logical focused-control identity and current vertex rectangles **before** clearing the existing SVG contents. Clear the stage SVG with set_inner_html(""), attach SvgRoot and construct Scene and its graph at saved vertex positions. Restore Scene::set_view(saved_view), wire the selection toolbar, restore its step selection, commit the new Demo state, then refocus the logically equivalent new control. Call refresh_layout() when the viewport changes. Preserve the same viewBox coordinates. This is not a transactional build-before-retire swap: if drawing fails after the clear, report via the existing error banner and recover on the next requested redraw.

Keyboard focus cannot persist on removed DOM elements. Follow the existing focused_button/focus_button (or focused_selector/focus) approach to identify the logical control and restore focus after replacement. Keep the screen-reader announcement/live region **outside** the cleared SVG so a step produces one meaningful announcement. Preserve input caret or IME state if the demo later accepts text entry; the initial Dijkstra demo need not support input editing during redraw.

7. Priority queue and termination

Queue storage is demo-owned. Display a priority-ordered summary with text cells or Markdown, not a new PriorityQueueNode. A binary heap with lazy duplicates is valid; stale entries are ignored when their vertex is settled or their queued distance is obsolete. Do not imply the summary order is the internal heap-array order.

Fixed queue viewport: place the queue in a rectangle with a fixed origin and size, independent of the graph and state-array layout, for **every** snapshot. Reserve exactly eight entry rows plus one overflow-summary row (e.g., "+3 more entries"); pad unused rows with blank display cells or leave them visually empty without shrinking the enclosing region. Precompute width hints for the longest queue string. The entire queue stays within its reserved rectangle, and its variable contents never change any graph vertex, table or connector geometry. This displays at most eight entries, while the algorithm keeps the complete queue.

Run until the queue is empty for a complete single-source solution, or stop on settling the selected target for a target-specific run. A disconnected destination must result in an explicit message such as “No route from A to F”, not an empty predecessor chain shown as success.

Verified: Scene::add_edge_with explicitly returns **Error::SelfLoopUnsupported** when from == to (src/scene/connector/mod.rs:214). Reject self-loops during demo validation with a clear explanation; the pure model may handle nonnegative self-loops independently, but do not try to render them using unsupported connectors. Directed parallel edges should remain valid and are tested.

8. Implementation sequence

Phase	Deliverable	Gate
1	Pure Dijkstra implementation and snapshot tests	Correctness against reference graph and edge cases
2	Text cells + cell-width hint	Unicode, sizing and accessible selections
3	Pure edge label placer, then SVG integration	≤8 candidates, ≤2 passes, 12-point hysteresis; stale retries
4	Per-edge focus	Concurrent final-path highlights; no style conflicts
5	Integrated demo and CDP regression suite	Reuse existing rebuild/focus/announcement pattern; fixed queue

9. Test and acceptance matrix

Test	Expected result
Reference example	Final distances and A→C→B→D→E→F total 13; reachable intermediate snapshot
Disconnected / source=target	Explicit unreachable narration / zero-cost source path
Zero weight, cycles, parallel edges	Correct finite results and deterministic trace
Ties and stale heap entries	Stable predecessor and no distance corruption
Overflow	checked_add error; no wrap or sentinel aliasing
Text Unicode	∞, — and labels render; accurate measurements and aria text
Edge labels	≤8 candidates, ≤2 passes, 11/12-point threshold and C→D/C→E collision
View rebuild	Existing clear/rebuild pattern; view, dragged positions, keyboard focus preserved
Keyboard / accessibility	Logical control focus restored; step narration; focus not colour-only
Final highlight	Multiple simultaneous edge focuses match predecessor route
Queue fixed region	Eight slots + overflow row; changing entries never moves vertices or tables
Fault recovery	Failed path/label update retried on next route update or drag release

CDP budget: approximately four or five browser tests, one behaviour per test: (1) text/accessibility, (2) edge-label placement and endpoint dragging, (3) multi-edge focus, (4) scene rebuild preserving view, vertex positions, focus and announcement, optionally (5) fixed queue-region geometry over all steps. Pure native tests cover algorithm edge cases, hard collisions, 11/12-point improvement, one-pass reconciliation, stale fault-injection/retry state, and WCAG contrast. Reuse shared CDP helpers and timing-based sharding.

10. Rejected expansions and completion rule

Do not introduce DijkstraNode, DistanceNode, PredecessorNode, WeightedVertexNode, PriorityQueueNode, a general heterogeneous-cell engine, an algorithm executor inside Scene, or arbitrary markdown/HTML embedded inside text cells.

Completion criterion: The demo computes data only in pure Rust, draws correct weighted connections and reachable snapshots, preserves pan/zoom, vertex positions and focus through established redraw handling, maintains a fixed queue region, retries stale edge writes on defined triggers, and introduces only generic library APIs.

Repository reference points (verified against main)

Version verified from Cargo.toml: **0.2.40**. Public Scene::move_node: src/scene/node/mod.rs:298; self-loop rejection: src/scene/connector/mod.rs:214. Relevant implementation locations: src/model/content/node_values.rs (Copy-bound formatting); src/model/content/labelling_style.rs (static row labels); src/scene/connector/connector_handle.rs (per-edge SVG handles); src/scene/scene_inner.rs (write_edge_path, redraw_edge); src/geometry/mod.rs (elbow_path_into); src/scene/toolbar/mod.rs (view, set_view); demo-app/src/huffman/mod.rs, demo-app/src/hamming/mod.rs and demo-app/src/crc32/mod.rs (focus-preserving rebuild pattern).

Repository: https://github.com/ChrisWhealy/svg-dom-graph